ResumeLens: A Four Stage Formal Language Pipeline for Résumé Qualification Screening

Daniel Bautista · Jeanfer Rivera · Samuel Rengifo

Computación y Estructuras Discretas III · Universidad Icesi · 2026-2

Abstract

ResumeLens applies four classical formal language models regular expressions, finite state transducers, finite automata and a context free grammar to turn unstructured résumé text into a validated, machine checkable candidate profile. The system does not rank candidates; it only verifies whether the qualifications it can detect satisfy a formally defined pattern for one of four professional profiles.

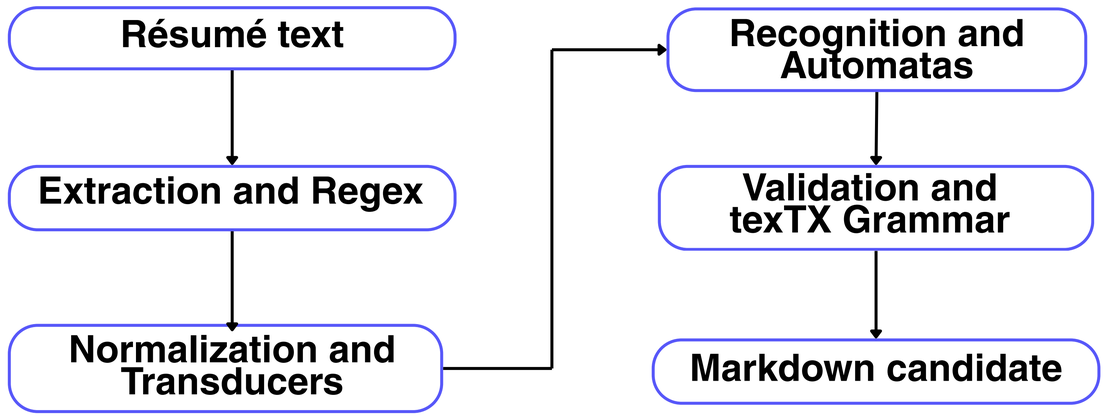
Introduction

The problem.
Résumés describe the same qualification in many forms like JS, Javascript and JavaScript are one skill. This ambiguity must be resolved formally.

Why formal languages.
We treat résumé screening as a chain of formal language problems, each matching a class of recognizer/transducer studied in this course. This gives the system a verifiable theoretical foundation every accept/reject decision traces back to a 5 or 7 tuple definition, not a heuristic.

Supported profiles: Full Stack Developer and Machine Learning Engineer (assignment defined), plus AI Engineer and Cloud Engineer (team defined).

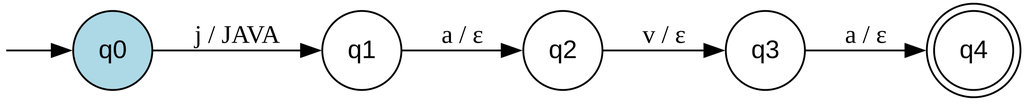
Methods



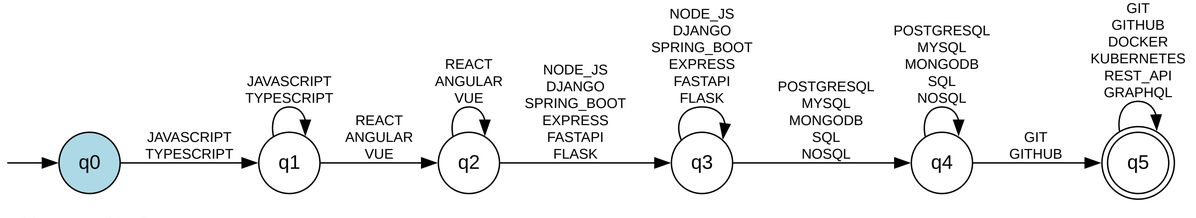
Database extraction: Detects database names in résumé text.

```
(?<![\w.+#])(?:PostgreSQL|Postgres|MySQL|SQLite|MongoDB|Mongo|Redis|Oracle|NoSQL|SQL) (?![\w+#])
```

Java normalization: Converts “java” into the canonical skill “JAVA”



FullStack recognition: Checks whether the candidate meets the required skill groups



Candidate grammar: Defines the structure of skill entries in the candidate profile

```
Skill:
    'skill' name=/[A-Z][A-Z0-9_]*/
;
```

Testing & deployment.
Streamlit runs all four stages on a pasted or uploaded résumé and renders ACCEPTED/REJECTED per profile

Discussion

LIMITATIONS

- **Catalogue bound coverage.** An unseen technology is reported as unrecognized, never invented correct behavior, but it means recall is capped by the transformation catalogue, not by the formal model itself.
- **Order sensitivity.** Stage 3 automata read groups in a fixed sequence (Stage 2's profile order), so a technically qualified résumé that happens to list backend before frontend is rejected for that strict pattern.
- **Syntactic, not semantic, validation.** textX guarantees the candidate profile is well formed; it does not verify that an e mail is deliverable or that a stated "3 years of experience" is numerically consistent with other fields.

POSSIBLE IMPROVEMENTS

- Grow the transformation catalogue and profile library beyond the four current profiles.
- Add an HTML visualization alongside the current Markdown output.